

Subsystem Decomposition & Architecture – Deliverable 4

Design Goals

The BILSEN Review System is an internal paper review platform developed for use by lab participants, including authors, reviewers, and coordinators, within a research environment. Its design goals were shaped by the specific needs of academic peer review. These needs include handling confidential paper submissions, supporting different user roles with distinct responsibilities, enforcing deadlines and conflict-of-interest (COI) rules, and maintaining a structured and traceable review workflow. In addition, the system must remain practical for everyday use by researchers while also interacting with external services such as AI-based review support and scholarly search tools. Among many possible software quality goals, reliability, usability, and fault tolerance were selected as the most important because they address these needs most directly and have the strongest effect on the effectiveness and continuity of the review process.

Reliability

For an academic review platform, dependable and accurate system behavior is essential. BILSEN supports important operations such as enforcing review deadlines, applying conflict-of-interest (COI) rules, and managing user roles correctly. The platform must ensure that only authorized users can access sensitive information and that submissions, reviewer assignments, and review records are stored safely without loss or inconsistency. Through strict backend checks and reliable database operations, BILSEN is designed to maintain the correctness and stability of the review process for every paper. Reliability was chosen as a primary design goal because academic review workflows depend heavily on trust. If permissions are handled incorrectly, reviews are lost, or assignments become inconsistent, the credibility of the entire review process may be damaged.

Usability

For an academic review platform, making user interactions simple and clear is highly important. BILSEN is used by different types of users, including authors, reviewers, and coordinators, each with different responsibilities throughout the review cycle. The system must present these responsibilities in an organized way so that users can easily understand what actions they need to take without confusion. By offering a unified dashboard, focused workflow sections, and clear status updates, BILSEN reduces the complexity of the review process and allows lab members to complete their tasks efficiently with minimal confusion and effort. Usability was selected as a key design goal because the platform will be used by people whose main focus is research and review tasks, not learning a complex system. If the interface is confusing or difficult to follow, users may make more mistakes, spend unnecessary time on simple actions, and experience delays in the review process.

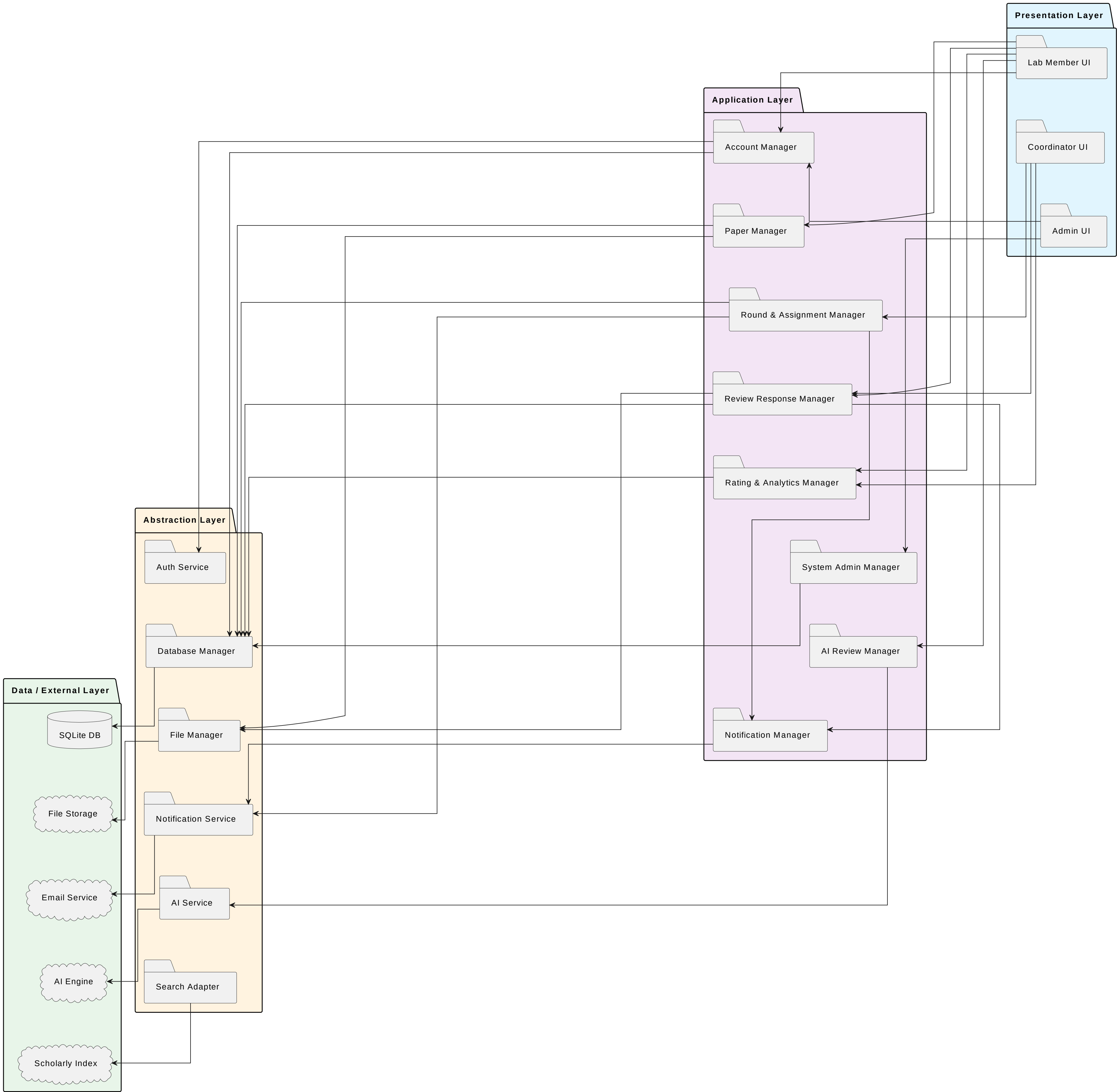
Fault Tolerance

BILSEN also depends on external services, especially the AI Inference Engine for automated review support and the Scholarly API for related-work analysis. Because of this, the system must be able to continue functioning even when these services are temporarily unavailable, slow, or unresponsive. When an external dependency fails, the platform should handle the situation in a controlled way by informing the user clearly and preventing the failure from affecting the rest of the system. In this way, essential tasks such as manual reviewing, paper tracking, and reviewer assignment can still continue without interruption. Fault tolerance was selected as a major design goal because some advanced features of the system rely on services outside the platform's direct control. The review process should therefore not come to a halt simply because an external AI or search service is unavailable for a limited period of time.

Tradeoffs

Selecting reliability, usability, and fault tolerance as the main design goals also required accepting several tradeoffs. Giving high priority to reliability means adding stricter validation, stronger permission checks, and more careful database handling, which improves correctness but also increases implementation complexity and development effort. Focusing on usability makes the system easier to understand for authors, reviewers, and coordinators, yet it requires additional work in interface design and workflow organization so that the platform remains clear without becoming oversimplified. Similarly, emphasizing fault tolerance helps the system continue operating when external services such as the AI Inference Engine or Scholarly API fail, but this also introduces extra architectural complexity because fallback behavior, error handling, and separation between core and external features must be designed carefully. Despite these tradeoffs, these three goals were considered the most suitable because they support the trustworthiness, practicality, and continuity required by an academic review platform.

BILSEN Review Management System - Enhanced Subsystem Decomposition



The BILSEN Review Management System follows a layered architecture consisting of four main layers: Presentation, Application, Abstraction, and Data / External. This structure separates user interaction, business logic, technical service access, and infrastructure resources, making the system easier to maintain, understand, and extend.

Presentation Layer

The Presentation Layer contains the user-facing interfaces of the platform. It is divided according to major user groups so that each interface focuses on the responsibilities of its own role.

Lab Member UI: Provides the main interface for regular lab participants. It supports account-related actions, paper-related tasks, review responses, analytics viewing, and access to AI-assisted review functions.

Coordinator UI: Supports the operational side of the review process. It is mainly used for managing rounds and assignments, monitoring responses, and viewing analytics about review progress.

Admin UI: Handles administrative operations in the system. It provides access to management functions related to accounts and overall platform control.

Application Layer

The Application Layer contains the main business logic of the platform. It coordinates workflows, applies system rules, and connects upper-layer requests to lower-layer technical services.

Round & Assignment Manager: Manages review rounds and reviewer assignment workflows. It acts as one of the central components of the review cycle and coordinates related supporting services.

Review Response Manager: Handles review submission and response-related processes. It supports the management of review actions together with notification-related functions.

System Admin Manager: Supports system-level administrative operations. It is responsible for backend control functions needed by the administrative side of the platform.

Rating & Analytics Manager: Responsible for ratings, summaries, and analytical views in the system. It processes data that helps users and coordinators observe review outcomes and activity.

Account Manager: Manages account-related operations of the system. It handles user-oriented account workflows and forwards authentication-related requests to the proper lower-layer services.

Paper Manager: Handles paper-related operations such as managing paper data and associated content. It coordinates persistence and file-related processes for submitted papers.

AI Review Manager: Handles AI-assisted review functionality in the system. It keeps AI-related operations separate from the rest of the core workflow logic.

Notification Manager: Coordinates notification-related actions in the application layer. It prepares notification requests and passes them to the corresponding service layer component.

Abstraction Layer

The Abstraction Layer isolates the application logic from implementation-specific technical details. It provides service boundaries for storage, authentication, communication, and external integrations.

Notification Service: Provides the technical mechanism for sending notifications. It connects application-layer notification requests to the external email infrastructure.

Database Manager: Handles communication with the persistent data store. It provides a common access point for storing, retrieving, and updating structured system data.

Auth Service: Provides authentication and authorization support for the platform. It separates identity-related technical logic from the application layer.

File Manager: Manages file-related technical operations. It is responsible for handling uploaded paper files and similar content resources.

Search Adapter: Provides an abstraction boundary for possible scholarly search integration with external indexing resources.

AI Service: Connects the platform to the external AI engine. It handles the technical side of AI-supported review operations.

Data / External Layer

The Data / External Layer contains the actual infrastructure resources and third-party systems used by the platform. These components perform storage and external service operations required by the system.

Email Service: Supports outgoing email-based notifications. It is used for actions such as invitations, reminders, and other communication events.

SQLite DB: Stores the structured data of the system. This includes user information, paper records, assignments, reviews, and related metadata.

File Storage: Stores uploaded files and document-related content. It is used for keeping paper files and similar resources outside structured database records.

Scholarly Index: Serves as the external scholarly search resource. It supports related-work or search-oriented functionality needed by the platform.

AI Engine: Provides external AI-based review support. It is used by the system when AI-assisted review functionality is triggered.